problem, because when implementing end-to-end encryption there are no session keys or record sequence numbers that can be used to safely implement a deterministic nonce.

Thankfully, COSE has a solution in the form of HKDF (hash-based key derivation function) that you used in chapter 11. Rather than using a key to directly encrypt a message, you can instead use the key along with a random nonce to derive a unique key for every message. Because nonce reuse problems only occur if you reuse a nonce with the same key, this reduces the risk of accidental nonce reuse considerably, assuming that your devices have access to an adequate source of random data (see section 12.3.2 if they don't).

To demonstrate the use of COSE for encrypting messages, you can use the Java reference implementation from the COSE working group. Open the pom.xml file in your editor and add the following lines to the dependencies section:[7]

```
<dependency>
  <groupId>com.augustcellars.cose</groupId>
  <artifactId>cose-java</artifactId>
  <version>1.1.0</version>
</dependency>
```

Listing 12.13 shows an example of encrypting a message with COSE using HKDF to derive a unique key for the message and AES-CCM with a 128-bit key for the message encryption, which requires installing Bouncy Castle as a cryptography provider. For this example, you can reuse the